

Lab 3: Architectural Justification — Self-Service Coffee Kiosk System

Name: Raksha | SRN: PES1UG24CS363 | Class: F

Architecture Selection

We chose Layered Architecture for the Self-Service Coffee Kiosk System. The system is split into three layers: a Presentation Layer (User Interface Component), a Business Layer (Order Manager, Payment Service and Receipt Printer Components) and a Data Layer (Database Component). Each layer only calls the layer directly below it through well-defined interfaces.

Reasons for the Choice

- **Small, fixed scope on a single device.** The kiosk offers only 3 coffee types, 2 sizes and one payment method, and runs as one application on one machine. Microservices would add service deployment, network calls and distributed data consistency for no gain, since no part of the kiosk needs to scale independently. Client-Server would make every order depend on a central server, a single point of failure. A layered design runs fully on the kiosk and maps directly onto the linear workflow: select → pay → print.
- **Hardware and menu changes stay isolated.** The kiosk depends on a touch screen, a card reader and a receipt printer, which may be replaced over time. Because the Order Manager only talks to the IReceipt and IPayment interfaces, swapping the printer model or card terminal changes only one component. Updating menu items or prices only changes data in the Data Layer, and redesigning screens does not touch business rules. Each layer can also be tested on its own with mock interfaces.

Security Advantage

Strict layering creates a controlled path to sensitive data. The UI can never read the database or handle raw card details directly; it can only call **IOrder**. Card data flows only inside the Payment Service to the card reader and is never stored in the Data Layer. The Order Manager also looks up the price from the database itself instead of trusting a price sent by the UI, so a tampered front end cannot change what the customer is charged.

Performance Benefit

All layers run in the same process on the kiosk, so calls between them are local method calls that take microseconds, not network requests. The small menu (6 price combinations) can be loaded once from the Data Layer and kept in memory by the Order Manager, so selection screens respond instantly to touch input. The only external call per order is card authorisation, which keeps the complete order cycle fast. The usual layered drawback, being unable to scale layers separately, does not apply because one kiosk serves one customer at a time.

Components and Interfaces

Interface	Provider → Consumer	Technology / data exchanged
IOrder	Order Manager → UI	In-process API call: getMenu(), placeOrder(type, size)
IPayment	Payment Service → Order Mgr	Payment API call: charge(amount) → approved / declined
IReceipt	Receipt Printer → Order Mgr	Hardware interface (ESC/POS over USB): printReceipt(order)
IMenuData	Database → Order Manager	Database queries (SQLite): getPrice(), saveOrder()